一、神经网络参数与卷积计算

1. 卷积输出尺寸计算公式：

$$\text{输出尺寸} = \frac{\text{输入尺寸} - \text{卷积核尺寸} + 2 \times \text{padding}}{\text{stride}} + 1$$

- 输入尺寸指的是 H × W（高度和宽度）
- padding: 填充的像素层数
- stride: 卷积步长

2. 卷积层参数量计算（不含 BatchNorm）：

$$\text{参数量} = (\text{卷积核高} \times \text{卷积核宽} \times \text{输入通道数}) \times \text{输出通道数} + \text{输出通道数（偏置项）}$$

示例：

- 输入尺寸：32 × 32 × 3
- 卷积核：5 × 5
- 输出通道数：6，padding=0，stride=1

$$\text{输出尺寸} = \frac{32 - 5 + 0}{1} + 1 = 28 \Rightarrow \text{输出为} 28 \times 28 \times 6$$

$$\text{参数量} = (5 \times 5 \times 3 + 1) \times 6 = 456$$

3. 池化操作（如 MaxPooling、AvgPooling）：

- 不涉及可学习参数
- 输出尺寸公式同卷积

二、正向传播（Forward Propagation）计算

- 神经元输出公式：

$$z = W \cdot x + b, \qquad a = f(z)$$

示例：

- 输入向量：x = [1, 2]
- 权重矩阵：W = [[0.1, 0.2], [0.3, 0.4]]
- 偏置：b = [0.5, 0.5]

计算：

$$z = [1 \times 0.1 + 2 \times 0.3 + 0.5, 1 \times 0.2 + 2 \times 0.4 + 0.5] = [1.2, 1.5]$$

$$a = \text{sigmoid}(z) \approx [0.7685, 0.8176]$$

四、PyTorch 常见结构与语法

python 复制 编辑

```python
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        return self.fc2(x)

# 训练过程:
for epoch in range(epochs):
    for data, target in loader:
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
```

五、常见激活函数

名称	表达式	特点
Sigmoid	$\frac{1}{1+e^{-x}}$	输出范围(0,1)，易饱和
Tanh	$\frac{e^x-e^{-x}}{e^x+e^{-x}}$	输出范围(-1,1)
ReLU	$\max(0,x)$	收敛快，易死神经元问题
LeakyReLU	x if $x>0$，否则 αx	缓解死神经元问题

六、常见损失函数总结

损失函数	用途	表达式说明
MSE	回归	$(y_{pred}-y_{true})^2$
CrossEntropyLoss	多分类	已内含 softmax
BCELoss	二分类	$-y\log(p)-(1-y)\log(1-p)$
SmoothL1Loss	稳健回归（目标检测中常见）	结合L1与L2损失

检测网络:

❶ YOLO 模型的主要思想是什么？优缺点有哪些？

YOLO（You Only Look Once）是一种端到端的目标检测方法，将目标检测问题转化为一个回归问题。它将输入图像划分为 S×S 的网格，每个网格预测若干个边界框及其所属类别概率。

优点：

检测速度快，可用于实时检测。

网络结构简单，训练端到端。

缺点：

对小目标和密集目标检测效果较差。

精度略低于两阶段方法（如 Faster R-CNN）。

❷ Faster R-CNN 的结构和工作流程是怎样的？

Faster R-CNN 是一种两阶段目标检测算法，主要包括以下几个部分：

卷积特征提取器（Backbone）：如 VGG、ResNet。

区域提议网络（RPN）：生成候选框（region proposals）。

RoI Pooling/Align 层：将候选框映射到统一大小的特征图。

分类和边界框回归分支：对每个 RoI 进行目标分类和边界框位置微调。

优点：

检测精度高，适合复杂场景。

RPN 替代传统的 Selective Search，速度提升明显。

❸ 比较 YOLO 与 Faster R-CNN 的异同。

项目	YOLO	Faster R-CNN
类型	单阶段（One-stage）	两阶段（Two-stage）
精度	中等	高
速度	快，实时性强	慢
特点	端到端回归，结构简单	RPN 生成候选框，结构复杂
小目标检测	效果差	表现更好

❹ Anchor（锚框）在目标检测中的作用是什么？

Anchor 是一种预设的参考边界框，用于预测不同大小和宽高比的目标。网络在每个位置上根据 anchor 调整预测结果，从而实现多尺度检测。

常见于 Faster R-CNN、YOLOv3、RetinaNet 等。

Anchor 的设计直接影响检测效果，尤其是对小目标。

❺ 什么是 IoU？目标检测中为何要使用 IoU？

IoU（Intersection over Union）是两个框的交并比，用于衡量预测框与真实框的重叠程度。

$$\mathrm{IoU} = \frac{\text{交集面积}}{\text{并集面积}}$$

通常将 IoU > 0.5 判断为正确检测。

- 用于评估检测结果的准确性。
- 在训练中用于匹配正负样本。

❻ 说明 YOLO 中如何将图像划分为网格并预测目标。

YOLO 将图像划分为 S×S 的网格，每个网格负责预测该区域中是否有目标中心点，并预测：

若干个边界框（B）的位置及其置信度（objectness score）。

每个边界框的类别概率（C 类）。

最终输出为 S×S×(B×5 + C) 的张量

❼ 什么是 NMS（非极大值抑制）？为什么需要它？

NMS（Non-Maximum Suppression）是一种后处理算法，用于移除重叠率高的冗余检测框。步骤：

按预测得分排序；

保留得分最高的框；

移除与之 IoU 大于阈值（如 0.5）的框；

重复直到没有剩余框。

原因：模型往往在一个目标附近输出多个框，NMS 可保留最佳预测。

深度学习期末复习笔记：程序分析题与程序填空题专辑

一、程序分析题（含激活函数、损失函数）

1. 简单神经网络输出

```
import torch

import torch.nn as nn


class Net(nn.Module):

    def __init__(self):

        super().__init__()

        self.fc1 = nn.Linear(2, 4)

        self.relu = nn.ReLU()

        self.fc2 = nn.Linear(4, 1)


    def forward(self, x):

        x = self.fc1(x)

        x = self.relu(x)

        x = self.fc2(x)

        return x


net = Net()

x = torch.tensor([[1.0, 2.0]])

y = net(x)

print(y.shape)
```

问题： 输出 `y.shape` 是什么？

答案： `torch.Size([1, 1])`，因为输入维度为 `[1,2]`，经过 `fc1(2→4)`、ReLU 和 `fc2(4→1)` 得到 `[1,1]`。

---

2. 损失函数逆向传播

```
x = torch.tensor([[0.5]], requires_grad=True)

target = torch.tensor([[1.0]])


loss_fn = nn.MSELoss()

loss = loss_fn(x, target)

loss.backward()

print(x.grad)
```

问题： `x.grad` 是多少？

答案： `tensor([[-1.0]])`，因为 MSE = (x - y)^2，对 x 求导得 2(x - y) = -1。

---

3. 训练过程参数是否更新

```
model = nn.Linear(1, 1)

optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

criterion = nn.MSELoss()


x = torch.tensor([[1.0]])

y = torch.tensor([[2.0]])


output = model(x)

loss = criterion(output, y)
```

```
optimizer.zero_grad()

loss.backward()

optimizer.step()


new_output = model(x)
```

问题： `new_output` 会变化吗？

答案： 会。如果初始输出小于 2，则会向上更新，优化器会使模型输出更接近 y。

---

4. Flatten 操作

```
x = torch.randn(10, 3, 28, 28)

x = x.view(x.size(0), -1)

print(x.shape)
```

问题： 输出是多少？

答案： `torch.Size([10, 2352])`，把图片每个样本展开为一维向量，适用于全连接层。

---

5. `.detach()` 问题

```
class MyModel(nn.Module):

    def __init__(self):

        super().__init__()

        self.fc = nn.Linear(10, 1)


    def forward(self, x):

        return self.fc(x).detach()


model = MyModel()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

x = torch.randn(5, 10)

y = torch.randn(5, 1)


optimizer.zero_grad()

output = model(x)

loss = ((output - y)**2).mean()

loss.backward()

optimizer.step()
```

问题： 参数会更新吗？
答案： 不会。`.detach()` 会断开计算图，导致无法进行梯度回传。

---

6. 激活函数 ReLU 与 Sigmoid

问题： 它们有何区别？
答案：ReLU：非线性强，不饱和，正向梯度好，但负区间梯度为 0 可能“死亡”；

Sigmoid：输出范围 (0,1)，平滑但饱和时梯度趋近 0，适合输出层。
结论： 隐藏层用 ReLU，输出层看任务（分类用 Sigmoid/Softmax，回归不加激活）。

---

7. 常见损失函数与适用任务

答案：

MSELoss: 回归任务

CrossEntropyLoss: 多分类任务

BCEWithLogitsLoss: 二分类任务（整合 Sigmoid 更稳定）

L1Loss: 对离群值更鲁棒

二、程序填空题

1. MLP 神经网络构建

```
self.net = nn.Sequential(
    nn.Linear(784, ____),
    ____,
    nn.Linear(128, 10)
)
```

答案： `128,nn.ReLU()`

---

2. 输出概率分布的网络

```
return ____
```

答案： `nn.functional.softmax(out, dim=1)`

---

3. 训练循环数据迭代填空

```
for i, (inputs, labels) in ____:
```

答案： `enumerate(dataloader)`

---

4. 卷积神经网络填池化层

```
self.pool = ____
```

答案： `nn.MaxPool2d(2, 2)`

---

5. 填损失函数

```
criterion = ____
```

答案： `nn.CrossEntropyLoss()` 或 `nn.MSELoss()`

---

6. ReLU 激活填空

```
relu = ____
```

答案： `nn.ReLU()`

---

7. 常见 CNN 填空题（补充）

```
self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)

self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

self.pool = nn.MaxPool2d(2, 2)
```

---

8. 分类模型最后一层填空

```
self.out = nn.Linear(128, 10)

...

return ____
```

答案： `F.log_softmax(x, dim=1)`